

DVD Rental Application

Run

The database connection is configured in DB_CONFIG in run.py.

`make run`

You can also truncate the existing records.

`make truncate`

Database schema

table	columns	notes
dvds	d_id PK AI, d_title VARCHAR(50), d_name VARCHAR(50), age_limit TINYINT, stock INT (default 2), cumul_rent_cnt INT, UNIQUE(d_title, d_name)	utf8mb4_bin on title/director so accent-sensitive
users	u_id PK AI, u_name VARCHAR(30), u_age INT, overdue INT, penalty_left INT, restricted TINYINT, cumul_rent_cnt INT	penalty state inlined for O(1) reads
ratings	(u_id, d_id) PK, rating TINYINT	one row per (user, DVD), upserts overwrite
borrowings	(u_id, d_id) PK	active loans only
reservations	d_id PK, u_id	at most one reservation per DVD (PK enforces)

Architecture

Single file run.py. Layers, top to bottom:

1. Connection layer (connection, cursor, fetch_one/fetch_all/fetch_required): cached connection, context-managed cursor, typed row helpers that cast the connector's union row type to tuple[Any, ...].
2. Format/parse helpers (fmt_avg, print_table, parse_int): table printer matches spec separator (- x80), avg rounds to 1 decimal then strips trailing .0.
3. Schema layer (DDL, create_schema, drop_schema, count_known_tables, load_csv): DDL as a tuple, executemany for batch CSV load.
4. Domain helpers (user_loads, require_dvd, require_user, read_id, do_borrow, bump_overdue_for_dvd, auto_checkout_reserver, reservation_flow): shared logic across menus.

5. Menu functions: one per spec section, names match the skeleton (`initialize_database`, `print_DVDs`, `insert_user`, etc.). Each opens its own cursor and commits at the end of a successful write.
6. Dispatch: MENU tuple of (label, callable) drives `main()`. Menu 13 (search directors) is None.

Implementation details

Penalty mechanics

- Every successful borrow bumps `overdue+=1` on all other users currently borrowing the same DVD.
- An unrestricted user crossing `overdue >= 5` flips to `restricted=1`, `penalty_left=2`. Already-restricted users keep accumulating overdue but stay flagged.
- When a restricted user attempts to borrow: error E13 and `penalty_left-=1`. Hitting 0 clears `restricted` and resets `overdue=0`. The user's next attempt is then evaluated normally.
- The E13 message prints the `penalty_left` value before decrement.

Reservation / auto-checkout

- Return path always sets `stock+=1` first, then attempts auto-checkout.
- `auto_checkout_reserver`: if a reservation exists, delete it, then check the reserver's `restricted` flag and `borrow < 3`. On success, `do_borrow` decrements `stock` (net 0, DVD transfers cleanly to reserver). On failure, `stock` stays at +1.
- Combined `(b+r <= 3)` invariant is preserved by the reservation flow's cap check, so the auto-checkout's borrow-count-only check is sufficient. Proof: at reservation registration `b+r <= 3`; deleting the reservation drops the count by 1 before borrowing.

User-based CF

- All matrix math in NumPy. Unrated cells filled with each user's own rating mean (or 0 if they have no ratings).
- Similarity is cosine on filled rows. Predictions are weighted sum of other users' ratings using sims as weights, divided by sim sum.
- Candidates: `target hasn't rated AND age_limit <= user.age`. Currently-borrowed-but-unrated DVDs included per spec.
- Tie-break: `np.lexsort((cand_d, -exp_rs))`. Primary key is the last argument (`-exp_rs` ascending = max first), secondary is `cand_d` ascending (smallest ID wins ties).

Impression

- It was a nice exercise to actually design a good database schema.